Template do Aluno: Mini Projeto "O Arquiteto Decisor"

Aluno: Gabryel Mariano da Silva | **Matrícula:** 2320877 **Link do Repositório GitHub (Scaffolding):** https://github.com/Scroduble/projeto-logitrack-doc.git

---

🟢 CICLO 1: Visão e Requisitos (Fase 1)

Preencher e entregar no AVA ao final do Ciclo 1.

1.1 Resumo do Cenário de Negócio - LOGITRACK

O LogiTrack – Sistema de Otimização de Rotas e Entregas tem como objetivo modernizar a operação de uma empresa de logística por meio da otimização de rotas em tempo real, gestão inteligente da frota e monitoramento contínuo das entregas. O sistema deve processar grandes volumes de dados geolocalizados, recalcular rotas dinamicamente diante de eventos como trânsito ou atrasos e fornecer previsões precisas de chegada (ETA).

O contexto de negócio envolve alta competitividade no setor logístico, onde eficiência operacional e cumprimento de prazos impactam diretamente custos e satisfação do cliente.

Os principais usuários são: Gestores logísticos (monitoramento e tomada de decisão); Motoristas (execução das rotas otimizadas); Equipe operacional (acompanhamento de entregas) e Sistemas corporativos integrados (estoque e ERP

1.2 Atributos de Qualidade (RNFs) Priorizados

1. Desempenho (Performance)

Justificativa:
A otimização de rotas deve ocorrer em tempo real, com respostas rápidas mesmo diante de múltiplas variáveis (tráfego, restrições de entrega, volume de pedidos). A lentidão comprometeria decisões operacionais e aumentaria custos logísticos. Do ponto de vista técnico, o sistema precisa lidar com algoritmos complexos e grande volume de dados geoespaciais, exigindo arquitetura preparada para baixa latência e alto throughput.

2. Escalabilidade

Justificativa:
O volume de entregas pode crescer significativamente em períodos sazonais. A arquitetura deve permitir crescimento horizontal sem perda de desempenho. Tecnicamente, isso implica distribuição de carga, serviços desacoplados e infraestrutura elástica. Sem escalabilidade, o sistema se tornaria um gargalo operacional.

3. Resiliência

Justificativa:
O sistema depende de redes móveis, APIs externas de mapas e sistemas corporativos. Falhas são inevitáveis. A arquitetura deve tolerar indisponibilidades temporárias, manter operação degradada controlada e evitar falhas em cascata. Do ponto de vista de negócio, paralisações afetam diretamente entregas e reputação da empresa.

4. Confiabilidade

Justificativa:
As informações de localização, status e previsão de chegada precisam ser precisas e consistentes. Erros podem gerar atrasos, retrabalho e perda financeira. Tecnicamente, exige controle de integridade de dados, consistência entre serviços e monitoramento contínuo.

5. Manutenibilidade

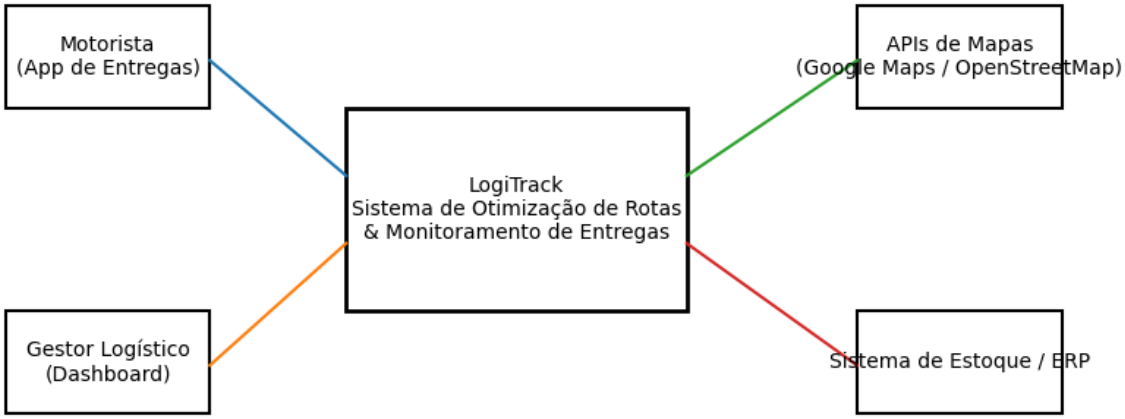
Justificativa:
O setor logístico é dinâmico e sofre alterações frequentes em regras de negócio, integrações e estratégias operacionais. A arquitetura deve permitir evolução controlada, facilidade de atualização e baixo acoplamento entre componentes. Uma arquitetura rígida elevaria custos e dificultaria adaptações futuras.

1.3 Diagrama de Contexto (C4 Nível 1)

O diagrama apresenta o **LogiTrack como uma caixa preta**, mostrando suas interações com os principais atores e sistemas externos:

Atores e sistemas externos:

- **Motorista** – utiliza o aplicativo para receber rotas e executar entregas.
- **Gestor Logístico** – acompanha operações e desempenho através de dashboards.
- **APIs de Mapas** (ex: Google Maps / OpenStreetMap) – fornecem dados de geolocalização e rotas.
- **Sistema de Estoque / ERP** – envia informações de pedidos e entregas para o LogiTrack.



Link digrams/:

1.4 Classificação da Estratégia

- **Classificação:** Balanceada
- **Justificativa:** A estratégia do LogiTrack é considerada balanceada por combinar tecnologias modernas, como inteligência artificial para otimização de rotas, com soluções já consolidadas no desenvolvimento de sistemas distribuídos. Essa abordagem reduz riscos técnicos enquanto permite inovação e evolução do sistema. Dessa forma, busca-se equilibrar desempenho, escalabilidade e confiabilidade. Conforme discutido por Roger S. Pressman, o desenvolvimento de software exige conciliar inovação com controle da complexidade e dos riscos.

🔵 CICLO 2: Blueprint e Decisões (Fase 2)

Preencher e entregar no AVA ao final do Ciclo 2.

2.1 Diagrama de Containers (C4 Nível 2)

[Insira aqui a imagem do seu Diagrama de Containers (C4 Nível 2). Este diagrama deve detalhar a estrutura interna do sistema, mostrando os principais containers (aplicações,

bancos de dados, filas, etc.) e suas interações. **Recomendado: Salvar o diagrama em /diagrams no GitHub e referenciar o link da imagem aqui.**]

2.2 Estilo Arquitetural Escolhido

[Descreva o estilo arquitetural principal que você adotou (ex: Microsserviços, Monolito Modular, Hexagonal, Event-Driven) e justifique sua escolha com pelo menos 3 trade-offs reais (prós e contras) em relação aos RNFs priorizados na Fase 1. Referencie o Capítulo 14 do Pressman sobre estilos arquiteturais, se pertinente].

2.3 Architecture Decision Record (ADR) Principal

[Escolha uma decisão arquitetural crucial que você tomou nesta fase (ex: Escolha da Tecnologia de Persistência, Estratégia de Comunicação entre Microsserviços) e documente-a como um ADR. **Recomendado: Criar o arquivo .md do ADR em /adrs no GitHub e referenciar o link aqui.**]

- **Título:** [Ex: Escolha da Tecnologia de Persistência]
- **Contexto:** [O problema ou necessidade técnica que levou a essa decisão. Qual era o cenário?]
- **Decisão:** [O que foi decidido. Qual tecnologia/abordagem foi escolhida?]
- **Justificativa:** [Por que esta opção venceu as alternativas? Quais foram os trade-offs considerados? Quais os impactos nos RNFs?]

🔴 CICLO 3: Cloud e Resiliência (Fase 3)

Preencher e entregar no AVA ao final do Ciclo 3.

3.1 Estratégia de Cloud e Implantação

[Descreva como sua arquitetura será implantada em um ambiente de nuvem (ex: AWS, Azure, GCP). Qual o modelo de serviço (IaaS, PaaS, Serverless)? Como o sistema será escalado e monitorado?]

3.2 Análise de Fragilidade e Mitigação

- **Ponto Frágil:** [Identifique a maior fraqueza ou risco da sua arquitetura em um cenário de produção (ex: falha de um serviço crítico, pico de tráfego inesperado, vulnerabilidade de segurança).]
- **Mitigação:** [Como você minimizaria esse risco? Quais estratégias de resiliência (ex: circuit breaker, retry, fallback, multi-região) seriam aplicadas?]

3.3 Parecer Técnico Final

[Resumo executivo (até 10 linhas) defendendo por que sua arquitetura é a melhor escolha para o cliente, considerando os requisitos de negócio, os RNFs e a estratégia de implantação. Qual o valor agregado da sua solução?]

🏆 BÔNUS: Evolução Arquitetural (Gamificação)

Opcional - Preencher apenas se houver melhoria estrutural documentada no GitHub.

[Descreva brevemente a melhoria arquitetural que você implementou no seu repositório GitHub para fins de bônus de nota. Referencie a nova ADR e o diagrama atualizado (se houver) no GitHub. Ex: Migração de um componente para Serverless, implementação de um padrão de resiliência, otimização de custo em cloud. **Lembre-se: a melhoria deve ser arquitetural, não apenas textual.**]

SURPREENDA-ME! :D

📚 Referências Bibliográficas

- **Pressman, R. S.** (2021). *Engenharia de Software: Uma Abordagem Profissional*. McGraw Hill. (Capítulos selecionados conforme o tema da aula).
- **Richards, M., & Ford, N.** (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
- **C4 Model for Software Architecture.** (s.d.). Disponível em: https://c4model.com/
- **Architecture Decision Records (ADRs).** (s.d.). Disponível em: https://adr.github.io/